

Experiment II — Advanced IaC Techniques

Field	Detail
Course outcome	CO5
Marks	5 marks (part of the 20 marks allotted to CO5)
Date of performance	_____
Marks awarded	_____

Aim

To explore advanced Infrastructure as Code capabilities by provisioning multiple interdependent virtual machines and by configuring network settings and security groups entirely through code.

Objectives

- To model a multi-tier network topology — VPC, subnets, gateway and routing — as code.
- To provision several virtual machines whose creation order is governed by dependencies.
- To express least-privilege firewall rules using security groups that reference one another.
- To refactor a flat configuration into reusable modules with defined inputs and outputs.
- To validate the design with both positive and negative connectivity tests.

Requirements

Hardware

Software

- Terraform CLI 1.6 or later
- AWS CLI v2, Git, Visual Studio Code
- Graphviz, for rendering the dependency graph produced by terraform graph
- An SSH client

Accounts and access

- The AWS account and IAM user from Experiment I, with permissions for EC2 and VPC
- An EC2 key pair created in the working region, for SSH access to the web tier

Theory

2.1 The Resource Dependency Graph

2.2 Meta-Arguments: count and for_each

2.3 Modules

2.4 Virtual Private Cloud Fundamentals

2.5 Security Groups and Least Privilege

Procedure

Step 1 — Design the topology on paper

Step 2 — Create the module directory structure

Step 3 — Provision the VPC and the subnets

Step 4 — Attach the internet gateway and configure routing

Step 5 — Author the security groups

Step 6 — Create the web tier using count

Step 7 — Create the dependent database instance

Step 8 — Wire the modules together in the root module

Step 9 — Inspect the dependency graph

Step 10 — Plan and apply the complete stack

Step 11 — Execute the verification tests

Step 12 — Demonstrate reusability, then destroy

Program and Configuration Listings

Listing 2.1 — modules/network/main.tf (network objects)

Listing 2.2 — modules/network/main.tf (security groups)

Listing 2.3 — modules/compute/main.tf

Listing 2.4 — root main.tf and module wiring

Expected Output

Output 2.1 — apply log showing the creation order

```
$ terraform apply
Plan: 11 to add, 0 to change, 0 to destroy.
Enter a value: yes

module.network.aws_vpc.main: Creating...
module.network.aws_vpc.main: Creation complete after 3s [id=vpc-0abc123]
module.network.aws_internet_gateway.igw: Creating...
module.network.aws_subnet.public: Creating...
module.network.aws_subnet.private: Creating...
module.network.aws_security_group.web: Creating...
module.network.aws_subnet.public: Creation complete after 2s
module.network.aws_subnet.private: Creation complete after 2s
module.network.aws_security_group.web: Creation complete after 4s
module.network.aws_security_group.db: Creating...      <-- waits for web SG
module.network.aws_security_group.db: Creation complete after 3s
module.compute.aws_instance.web[0]: Creating...      <-- both web servers
module.compute.aws_instance.web[1]: Creating...      start together
module.compute.aws_instance.db: Creating...
module.compute.aws_instance.web[1]: Creation complete after 33s
module.compute.aws_instance.web[0]: Creation complete after 34s
```

```
module.compute.aws_instance.db: Creation complete after 35s
```

```
Apply complete! Resources: 11 added, 0 changed, 0 destroyed.
```

```
Outputs:
db_private_ip = "10.0.2.15"
web_public_ips = [ "13.234.56.78", "13.234.56.91" ]
```

Output 2.2 — scaling test after changing web_count to 3

```
$ terraform plan -var 'web_count=3'

# module.compute.aws_instance.web[2] will be created
+ resource "aws_instance" "web" {
+   instance_type = "t2.micro"
+   tags         = { "Name" = "web-3", "Tier" = "web" }
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

My Experiment Output

Observation Table

Complete this table with the values observed during your own session.

Test	Method	Expected	Observed
Creation order		Network before compute	
Web reachability		HTTP 200 from both	
SSH restriction		Connection times out	Timed out after __ s
Database isolation		Refused or timed out	
Tier-to-tier access		Connection succeeds	
Reusability		1 to add, 0 to change	
Teardown order		Instances before subnets	

Result

A two-tier network topology consisting of a VPC, a public and a private subnet, an internet gateway, routing and two security groups, together with three interdependent virtual machines, ...

Precautions

- Set the admin CIDR to the laboratory's actual public address. Leaving SSH open to 0.0.0.0/0 will cause the configuration to be rejected during evaluation and exposes the instance to automated attacks within minutes.
- Place the database in the private subnet. An instance in a public subnet with a restrictive security group is still one misconfiguration away from exposure.
- Do not embed the key pair file or its contents in the configuration; reference the key by name only.
- Verify that every subnet CIDR lies within the VPC CIDR and that the two subnets do not overlap.